

Aristotle University of Thessaloniki
Department of Electrical and Computer Engineering
Electronics Division

Αθανάσιος Κωνσταντής (10537)

Hardware II

Δημιουργία module πολλαπλασιασμού αριθμών κινητής υποδιαστολής με τη
χρήση SystemVerilog

Η εργασία αφορά ένα module που εκτελεί την πράξη του πολλαπλασιασμού για αριθμούς κινητής υποδιαστολής των 32bit. Το βασικό διάγραμμα του module φαίνεται στην παρακάτω εικόνα

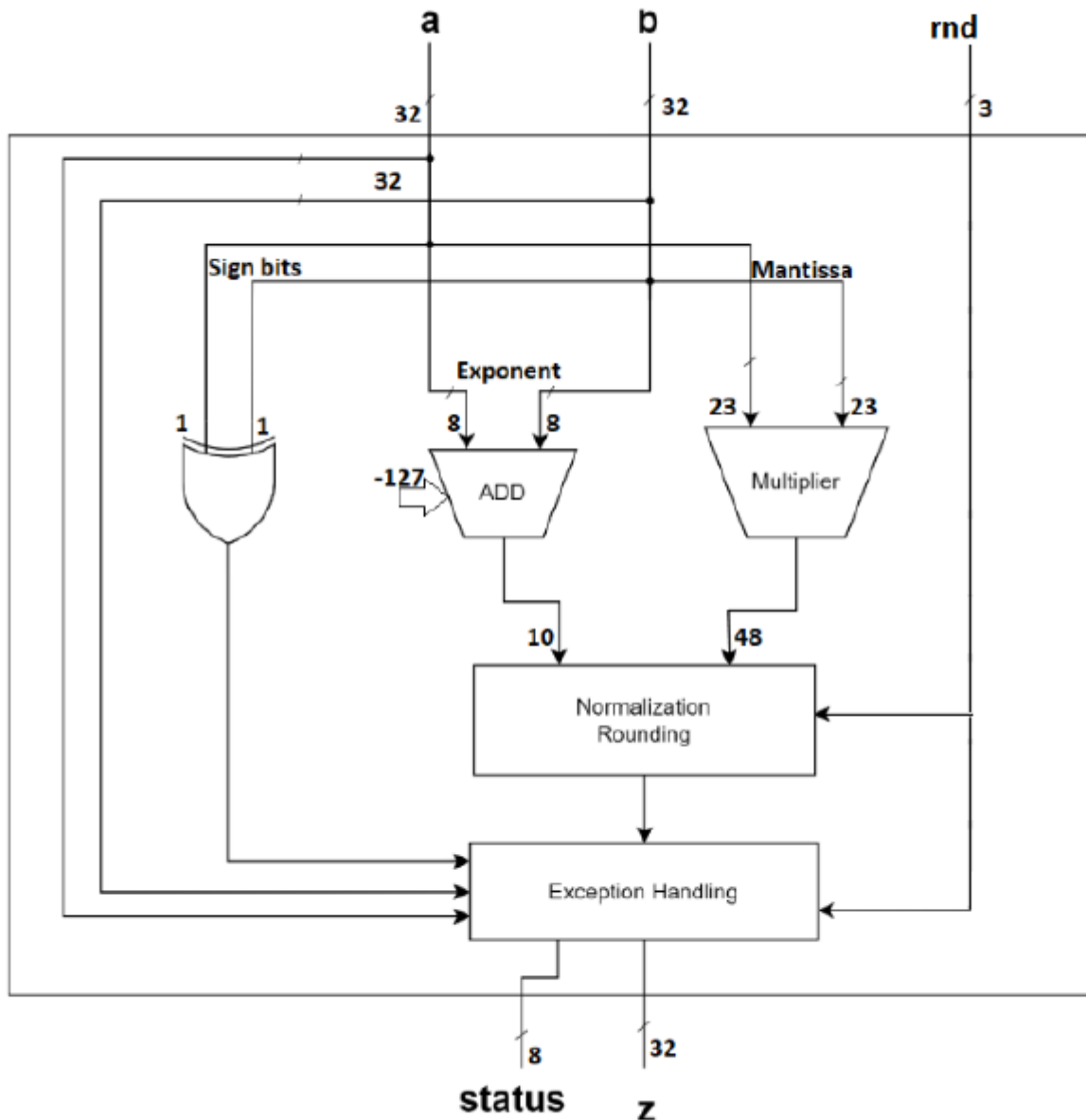


Figure 1: Μπλοκ διάγραμμα του main module του πολλαπλασιαστή

1 Main Module

Το main module εμπεριέχει τρία submodules τα οποία είναι το normalization, rounding και το exception module. Επίσης, έχει 3 inputs (δύο 32-bit αριθμοί a και b , καθώς και ο 3-bit αριθμός rnd) και 2 outputs (το αποτέλεσμα 32-bit z και ένας 8-bit αριθμός που δείχνει το status).

Αρχικά, υλοποιούμε τον υπολογισμό του προσήμου του τελικού αριθμού μέσω μιας πράξης XOR στα MSB των a και b .

Έπειτα, υλοποιούμε την πρόσθεση των εκθετών των δύο αριθμών μέσω των απλών συμβόλων '+', '-' που μας προσφέρει η SystemVerilog. Υπογραμμίζεται ότι στη συγκεκριμένη περίπτωση, η

SystemVerilog συνθέτει ένα κύκλωμα αθροιστή που ίσως να μην είναι ιδανικό. Γι'αυτό σε κάθε περίπτωση καλό είναι να έχουμε μια πλήρη περιγραφή του αθροιστή μας και να μην χρησιμοποιούμε τα σύμβολα αυτά. Παρ'όλα αυτά, στη συγκεκριμένη περίπτωση μας ενδιαφέρει μόνο το αποτέλεσμα του simulation και όχι αυτό της σύνθεσης, γι'αυτό παραμελήθηκε η παραπάνω ένσταση.

Συνεπώς, γνωρίζοντας ότι οι εκθέτες των αριθμών βρίσκονται από το bit 30 ως το bit 23, υλοποιούμε την πρόσθεση των δυο 8-bit αριθμών και αφαιρούμε από αυτούς το bias. Καταλήγουμε με έναν αριθμό που έχει το πολύ 9 bits. Παρ'όλα αυτά, η πράξη που εκτελούμε είναι πράξη προσημασμένων αριθμών (μπορεί δηλαδή το αποτέλεσμα να είναι τελικά αρνητικό), άρα προσθέτουμε 1 bit ακόμη για το πρόσημο. Τελικά, έχουμε ένα διάνυσμα 10 bit.

Τέλος, υλοποιούμε και τον πολλαπλασιαστή με τον ίδιο απλό τρόπο, χρησιμοποιώντας το σύμβολο `">*` για να πολλαπλασιάσουμε τις Mantissa του κάθε αριθμού, η οποία γνωρίζουμε ότι ξεκινά από το bit 22 έως το bit 0. Για να έχουμε όμως σωστό αποτέλεσμα πρέπει να πολλαπλασιάσουμε δυο 24-bit αριθμούς, όπου το MSB τους θα είναι το 1 και τα υπόλοιπα bits θα είναι η Mantissa των a και b αντίστοιχα. Αυτό είναι λογικό μιας και η Mantissa είναι το υπόλοιπο ενός δυαδικού αριθμού που αναπαρίσταται σε scientific notation, αν του αφαιρέσουμε το MSB του (δηλαδή την μονάδα πριν από την υποδιαστολή). Για να εκτελέσουμε τον πολλαπλασιασμό μεταξύ των Mantissa πρέπει όμως να ξαναεμφανίσουμε αυτό το MSB που χάθηκε κατά τη μετατροπή του αριθμού σε floating point αναπαράσταση. Οπότε, έχοντας πολλαπλασιασμό διανυσμάτων 24-bit, το αποτέλεσμα είναι ένας αριθμός πλάτους 48-bit.

Το main module έχει επίσης το overflow και underflow detection το οποίο όμως θα εξηγηθεί παρακάτω.

2 Normalization Module

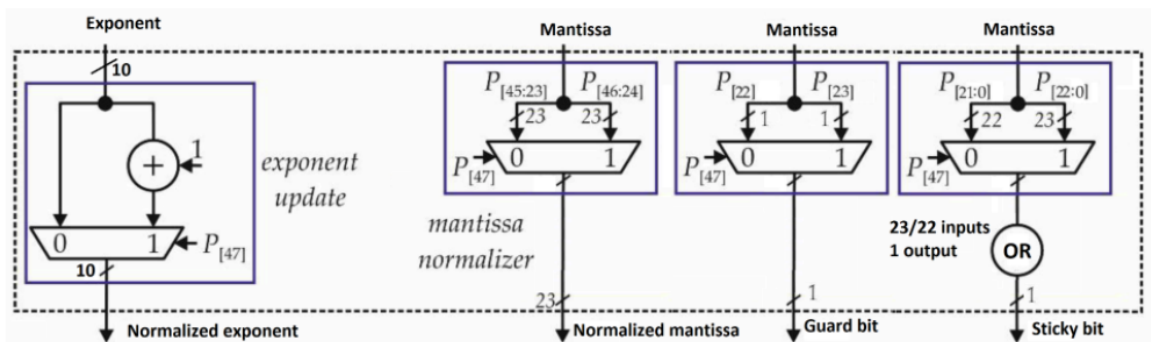


Figure 2: Μπλοκ διάγραμμα του Normalization Module

Το Normalization Module είναι απλό στην υλοποίηση. Έχει εισόδους 2 σημάτων, το 10-bit αποτέλεσμα της πρόσθεσης των εκθετών και το 48-bit αποτέλεσμα του πολλαπλασιασμού των Mantissa. Οι εξοδοί του είναι 4, το normalized exponent (10-bit), η normalized mantissa (23 bit) και τα guard και sticky bits. Ουσιαστικά όλα εξαρτώνται στο αν το MSB του διανύσματος P είναι 1. Σε περίπτωση που είναι, γνωρίζουμε ότι η υποδιαστολή βρίσκεται ανάμεσα στο bit 46 και 45, άρα πρέπει να προσθέσουμε 1 στον εκθέτη και να "κόψουμε" το αποτέλεσμα στα 23 bits έτσι ώστε να έχουμε την καινούρια μας Mantissa, αγνοώντας και το MSB που δεν εμπεριέχεται στην Mantissa. Αλλιώς δεν χρειάζεται να προσθέσουμε κάτι στον εκθέτη και απλά εκτελούμε το κόψιμο με αρχή το bit 45 αντί για το 46. Το Guard bit επίσης πρόκειται για το bit στα δεξιά του τελευταίου bit της Mantissa μας και το sticky bit είναι το αποτέλεσμα της πράξης OR σε όλα τα στοιχεία του υπολοίπου, "κομμένου" διανύματος.

Η υλοποίηση έγινε με τον τελεστή συνθήκης `'?:'` μιας και επρόκειτο για απλούς πολυπλέκτες εισόδου select του ενός bit.

3 Rounding Module

Το Rounding Module έχει εισόδους τις εξόδους του Normalization, το σήμα rnd και το αποτέλεσμα του προσήμου, και εξόδους το 25-bit αποτέλεσμα στρογγυλοποίησης της Mantissa, το 10-bit αποτέλεσμα στρογγυλοποίησης του εκθέτη και ένα flag 1-bit που γίνεται 1 μόνο όταν έχουμε στρογγυλοποίηση.

Αρχικά, ελέγχουμε αν τα guard και sticky bits είναι μηδενικά. Στην περίπτωση αυτή έχουμε ολόκληρο τον αριθμό μας και δεν υπάρχει ανάγκη για στρογγυλοποίηση. Εδώ το flag inexact μας παίρνει την τιμή 0. Σε κάθε άλλη περίπτωση, παίρνει την τιμή 1.

Πριν εξηγήσουμε πώς γίνεται η στρογγυλοποίηση, καλό είναι να πούμε ότι έχουμε δημιουργήσει ένα αρχείο globals.sv όπου έχουμε ένα package. Το package αυτό έχει μέσα ένα enum που αντιστοιχίζει τις τιμές των bits που μπορεί να πάρει το διάνυσμα rnd με τα ονόματα που μας δίνονται στον Πίνακα 4 του pdf. Το enum αυτό χρησιμοποιείται σε πολλά σημεία του κώδικα για καλύτερη αναγνωσιμότητα. Το package globals έχει και ένα ακόμη enum το οποίο θα εξηγηθεί στη συνέχεια.

Επίσης είναι σημαντικό να καταλάβουμε ακριβώς πώς δουλεύουν τα guard και sticky bits. Όταν το guard ή sticky bit έχει την τιμή 1, αυτό σημαίνει απλά ότι το κομμάτι από την Mantissa που "κόψαμε" ήταν διάφορο του μηδενός. Η τελική Mantissa μας δεν περιέχει ούτε το guard ούτε το sticky bit. Συνεπώς μπορούμε να καταλάβουμε ότι όταν και τα 2 αυτά bits είναι 0, δεν χάνουμε καθόλου πληροφορία και άρα δεν χρειάζεται στρογγυλοποίηση.

Τώρα που καταλαβαίνουμε την χρησιμότητα των 2 αυτών bits, είναι η ώρα να εξηγήσουμε πώς γίνεται η στρογγυλοποίηση για κάθε τιμή του rnd.

Επίσης θα πούμε το guard bit = G και το sticky bit = S για συντομία.

- IEEE_near
 - Αν $G = 0$ και $S = 1$, σημαίνει ότι ο αριθμός βρίσκεται πιο κοντά στο 0 απ'ότι στο 1, άρα κάνουμε truncate
 - Αν $G = 1$ και $S = 1$, σημαίνει ότι ο αριθμός βρίσκεται πιο κοντά στο 1 απ'ότι στο 0, άρα προσθέτουμε 1 στην Mantissa
 - Αν $G = 1$ και $S = 0$, η απόσταση από το 0 και το 1 είναι ίση, άρα πρέπει να κάνουμε τον αριθμό άρτιο. Συνεπώς, αν το LSB είναι 1 πρέπει να προσθέσουμε 1 στην Mantissa, ενώ αν είναι 0 απλά κάνουμε truncate.
- IEEE_zero
 - Πάντα κάνουμε στρογγυλοποίηση προς το 0, άρα πάντα κάνουμε απλό truncate.
- IEEE_pinf
 - Η στρογγυλοποίηση πάντα γίνεται προς το $+\infty$, οπότε το πρόσημο το αριθμού παίζει ρόλο. Αν ο αριθμός είναι θετικός, προσθέτουμε στην Mantissa 1. Αλλιώς κάνουμε truncate.
- IEEE_ninf
 - Η στρογγυλοποίηση πάντα γίνεται προς το $-\infty$, οπότε το πρόσημο το αριθμού παίζει ρόλο. Αν ο αριθμός είναι θετικός κάνουμε truncate. Αλλιώς προσθέτουμε στην Mantissa 1.
- near_up
 - Αν $G = 0$ και $S = 1$, αυτό σημαίνει ότι ο αριθμός είναι πιο κοντά στο 0 και άρα κάνουμε truncate.
 - Αν $G = 1$ και $S = 1$, αυτό σημαίνει ότι ο αριθμός είναι πιο κοντά στο 1 άρα προσθέτουμε στην Mantissa το 1.

- Αν $G = 1$ και $S = 0$, ο αριθμός ισαπέχει από τα 0 και 1, άρα στην περίπτωση αυτή το πρόσημο έχει σημασία. Αν ο αριθμός είναι θετικός προσθέτουμε στην Mantissa + 1, αλλιώς κάνουμε truncate.

- away_zero

- Πάντα στρογγυλοποιούμε μακριά από το 0, άρα προσθέτουμε στην Mantissa πάντα 1.

Τέλος, σε περίπτωση που το MSB της rounded Mantissa είναι 1, πρέπει να κάνουμε shift bitwise το διάνυσμα στα δεξιά κατά μια θέση, προσθέτοντας στον εκθέτη μια μονάδα.

4 Exception Handling Module

Το Exception Handling Module διαχειρίζεται τις οριακές συνθήκες. Έχει εισόδους τα αρχικά a,b διανύσματα 32-bit, το 32-bit διάνυσμα z_calc (το οποίο θα εξηγήσουμε πώς προκύπτει παρακάτω), το rnd σήμα και τα overflow, underflow, και inexact bits. Οι έξοδοι του είναι κάποια status bits και το αποτέλεσμα z του πολλαπλασιασμού.

Αρχικά δημιουργούμε το enum και τις δυο συναρτήσεις που ζητούνται από την εκφώνηση, σύμφωνα με τον πίνακα 1 του αρχείου pdf. Σε αυτό το στάδιο ακολουθήθηκαν ακριβώς οι οδηγίες της εκφώνησης και αρα δεν θα περιγραφεί αναλυτικά.

Έπειτα, δημιουργούμε ένα always_comb block όπου τσεκάρουμε για τις οριακές συνθήκες και ενημερώνουμε τα status bits ανάλογα.

- (Zero x Zero) , (Zero x Norm), (Norm x Zero)
 - Στην περίπτωση αυτή, το αποτέλεσμα πάντα είναι 0. Οπότε επιστρέφουμε το Zero με το πρόσημο που υπολογίσαμε και σηκώνουμε το zero flag.
- (Zero x Inf) , (Inf x Zero)
 - Στην περίπτωση αυτή πρέπει να επιστραφεί NaN, οπότε επιστρέφουμε το INF και σηκώνουμε το NaN flag και το inf flag
- (Inf x Inf) , (Inf x Norm), (Norm x Inf)
 - Το αποτέλεσμα εδώ είναι το Inf με το πρόσημο που έχει υπολογιστεί. Άρα το επιστρέφουμε σηκώνοντας το inf flag.
- (Norm x Norm)
 - Αν έχουμε overflow
 - * IEEE_zero
 - Πάντα κάνουμε στρογγυλοποίηση προς το μηδέν, άρα επιστρέφουμε τον μέγιστο αριθμό που μπορούμε και σηκώνουμε τα huge και inexact flags
 - * near_up, IEEE_near, away_zero
 - Σύμφωνα με τεστ που κάναμε, πρέπει να κάνουμε στρογγυλοποίηση πάντα προς τα πάνω με αυτά τα rounding modes. Αρα επιστρέφουμε Inf και σηκώνουμε τα inexact, inf και huge flags
 - * IEEE_pinf
 - Πάντα κάνουμε στρογγυλοποίηση προς το $+\infty$ άρα το πρόσημο του αριθμού παίζει ρόλο. Αν ο αριθμός είναι θετικός, επιστρέφουμε Inf με τα κατάλληλα flags. Αλλιώς επιστρέφουμε τον Max_Norm.
 - * IEEE_ninf

- Πάντα κάνουμε στρογγυλοποίηση προς το $-\infty$ άρα το πρόσημο του αριθμού παίζει ρόλο. Αν ο αριθμός είναι θετικός, επιστρέφουμε `Max_Norm` με τα κατάλληλα flags. Αλλιώς επιστρέφουμε `Inf`
- Αν έχουμε underflow
 - * `away_zero`
 - Αφού πάντα στρογγυλοποιούμε μακριά από το μηδέν, άρα επιστρέφουμε το `Min_Norm` σηκώνοντας τα `tiny` και `inexact` flags
 - * `near_up,IEEE_near,IEEE_zero`
 - Σε αυτή την περίπτωση στρογγυλοποιούμε προς τα κάτω και επιστρέφουμε το `Zero` σηκώνοντας τα `tiny`, `inexact`, `zero` flags
 - * `IEEE_pinf`
 - Πάντα κάνουμε στρογγυλοποίηση προς το $+\infty$ άρα το πρόσημο του αριθμού παίζει ρόλο. Αν ο αριθμός είναι θετικός, επιστρέφουμε το `Min_Norm` με τα κατάλληλα flags. Αλλιώς επιστρέφουμε `Zero`.
 - * `IEEE_ninf`
 - Πάντα κάνουμε στρογγυλοποίηση προς το $-\infty$ άρα το πρόσημο του αριθμού παίζει ρόλο. Αν ο αριθμός είναι θετικός, επιστρέφουμε το `Zero` με τα κατάλληλα flags. Αλλιώς επιστρέφουμε το `Min_Norm`.
- Σε κάθε άλλη περίπτωση, δεν έχουμε edge case και άρα επιστρέφουμε τον αριθμό `z_calc` με το `inexact` flag που δημιουργήθηκε από τα προηγούμενα modules.

5 Overflow & Underflow

Στο Main Module πρέπει να ελέγξουμε τις εξόδους του Rounding Module και σε περίπτωση που έχουμε overflow ή underflow να κάνουμε raise τα κατάλληλα σήματα. Ουσιαστικά, ο εκθέτης της εξόδου του Rounding Module είναι ένας προσημασμένος αριθμός 10-bit. Ο μέγιστος εκθέτης που μπορεί να έχει ένας floating point αριθμός 32-bit είναι το $(11111110)_2 = (254)_{10}$. Ο ελάχιστος εκθέτης είναι φυσικά το 1. Άρα, αν ο εκθέτης που έχει ως έξοδο το Rounding Module είναι μεγαλύτερος του 254 έχουμε overflow, ενώ αν είναι μικρότερος του 1 έχουμε underflow.

Τέλος, δημιουργούμε το σήμα `z_calc` και το σήμα `status`.

6 Testbenches

Για τα Testbenches φτιάξαμε δυο αρχεία **tb1.sv** και **tb2.sv**.

Στο πρώτο testbench χρησιμοποιούμε τη συνάρτηση *\$urandom* ώστε να παίρνουμε για τα `a` και `b` τυχαία διανύσματα των 32 bits. Το κάνουμε αυτό για 150 συνδιασμούς των `a` και `b` και για κάθε έναν από αυτούς τους συνδιασμούς τεστάρουμε όλα τα διαφορετικά rounding modes. Σε περίπτωση που το αποτέλεσμα μας είναι διαφορετικό από το αποτέλεσμα της συνάρτησης **multiplication** που μας δώθηκε, κάνουμε print πολλές πληροφορίες και ενημερώνουμε ότι υπήρξε κάποιο λάθος.

Σημειώνεται ότι σε αυτό το κομμάτι βρέθηκαν μερικά λάθη στον κώδικα και διορθώθηκαν. Ο συγκεκριμένος τρόπος για debugging είναι αρκετά αποδοτικός. Παρ'όλα αυτά, ο τελικός κώδικας δεν έχει λάθη και δεν πρέπει να τυπώνει τίποτα.

Για το δεύτερο κομμάτι έπρεπε να τεστάρουμε τα edge cases και για αυτό δημιουργήσαμε ένα enum type όπως προτείνεται από την εκφώνηση. Το type αυτό το δημιουργήσαμε στο package του **globals.sv** σε περίπτωση που χρειαζόταν να το κάνουμε access σε διαφορετικά αρχεία.

Σε παρόμοια φιλοσοφία με το πρώτο testbench, δημιουργούμε δυο εμφωλισμένες for loops που τεστάρουν κάθε δυνατό συνδιασμό edge cases (144 combinations) και κάνει print διάφορες πληροφορίες σε περίπτωση που βρεθεί σφάλμα.

Τα αποτελέσματα και των δυο testbenches βρίσκονται στο τέλος του pdf αρχείου.

7 Assertions

Ο κώδικας για τα Assertions μας είναι στο αρχείο **assertions.sv**. Ο κώδικας ακολουθεί την ίδια φιλοσοφία με τον κώδικα που μας δώθηκε στο pdf να μελετήσουμε ώστε να μάθουμε πώς να δημιουργήσουμε Assertions. Η εργασία αυτή γράφτηκε πριν μελετηθούν στο μάθημα τα Assertions.

Αρχικά πρέπει να βρούμε ποια σήματα δεν μπορούν να σηκώνονται ταυτόχρονα. Θα ελέγξουμε όλα τα σήματα ανά δυο για να δούμε αν μπορούν να είναι σηκωμένα ταυτόχρονα. Για τη μελέτη αποκλείουμε τα σήματα 'Unused', 'Div by 0' τα οποία γνωρίζουμε ότι πρέπει να είναι πάντα low.

zero	inf	×
zero	invalid	×
zero	tiny	✓
zero	huge	×
zero	inexact	✓
inf	invalid	×
inf	tiny	×
inf	huge	✓
inf	inexact	✓
invalid	tiny	×
invalid	huge	×
invalid	inexact	×
tiny	huge	×
tiny	inexact	✓
huge	inexact	✓

Οι συνδιασμοί με (✓) είναι επιτρεπτοί ενώ αυτοί με (×) όχι. Συνεπώς πρέπει να δημιουργήσουμε 9 properties που ελέγχουν αν έχουμε τους 9 πάνω συνδιασμούς, και άλλα 2 που ελέγχουν αν τα σήματα 'Unused', 'Div by 0' έχουν υψωθεί, και να τσεκάρουμε ότι αυτό δεν συμβαίνει μέσω των Assertions.

Για το δεύτερο κομμάτι των Assertions, ακολουθήσαμε ακριβώς τις οδηγίες που δίνονται στο pdf για την δημιουργία των properties που μας ζητούνται και άρα δεν θα κάνουμε κάποια ανάλυση της λογικής.

Ο κώδικας με τα Assertions βρίσκεται στο αρχείο **assertions.sv** το οποίο κάνοντας compile με κατάλληλα flags τρέχει για το πρώτο ή το δεύτερο ζητούμενο.

Συγκεκριμένα, χρησιμοποιούμε τις εντολές:

```
vlog -sv assertions.sv +define+immediate_assertions
vlog -sv assertions.sv +define+concurrent_assertions
```

αντίστοιχα για κάθε ζητούμενο.

8 Διαγράμματα Προσομοίωσης

8.1 Testbench 1

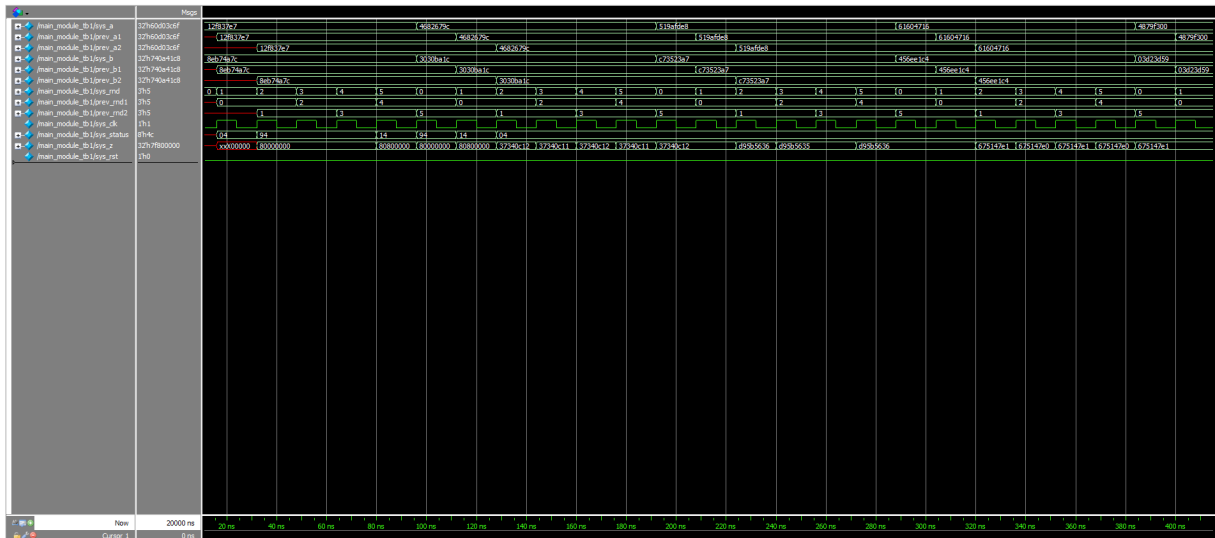


Figure 3: Διάγραμμα χρονισμού για random input

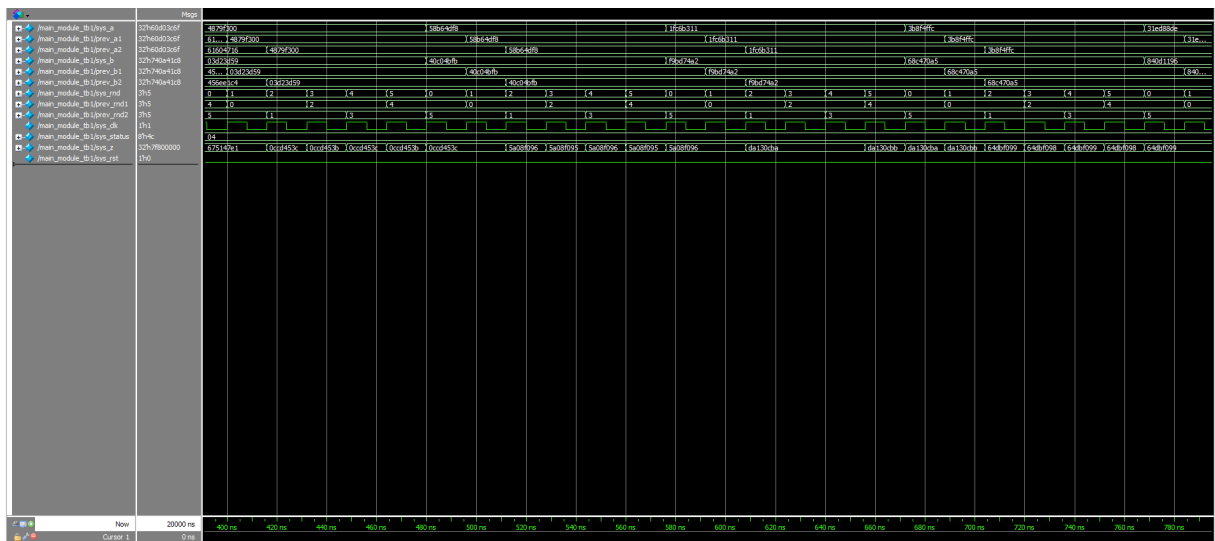


Figure 4: Διάγραμμα χρονισμού για random input



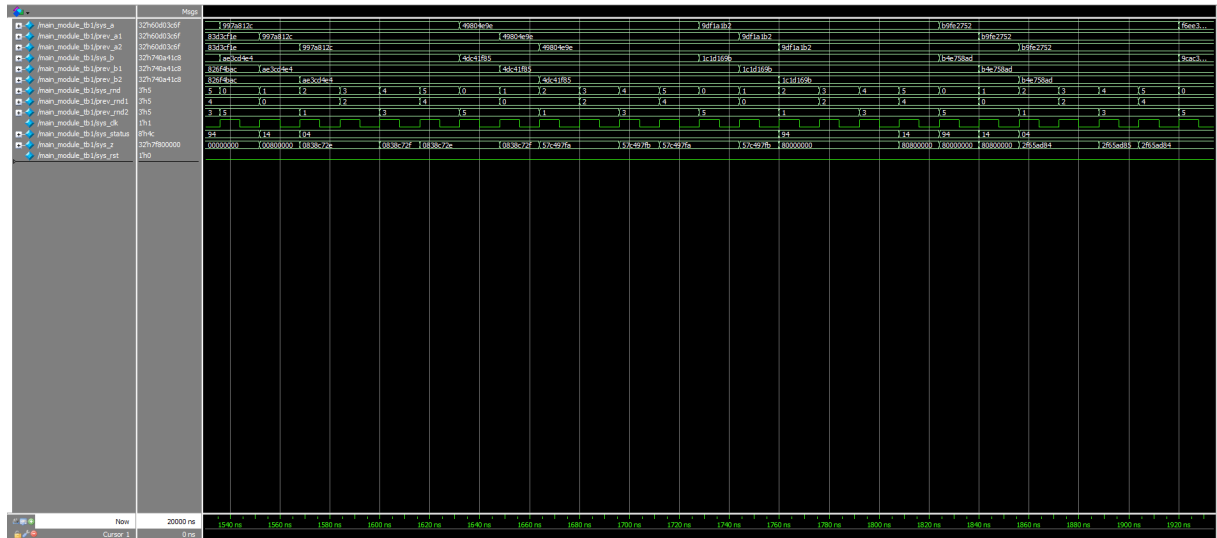


Figure 7: Διάγραμμα χρονισμού για random input

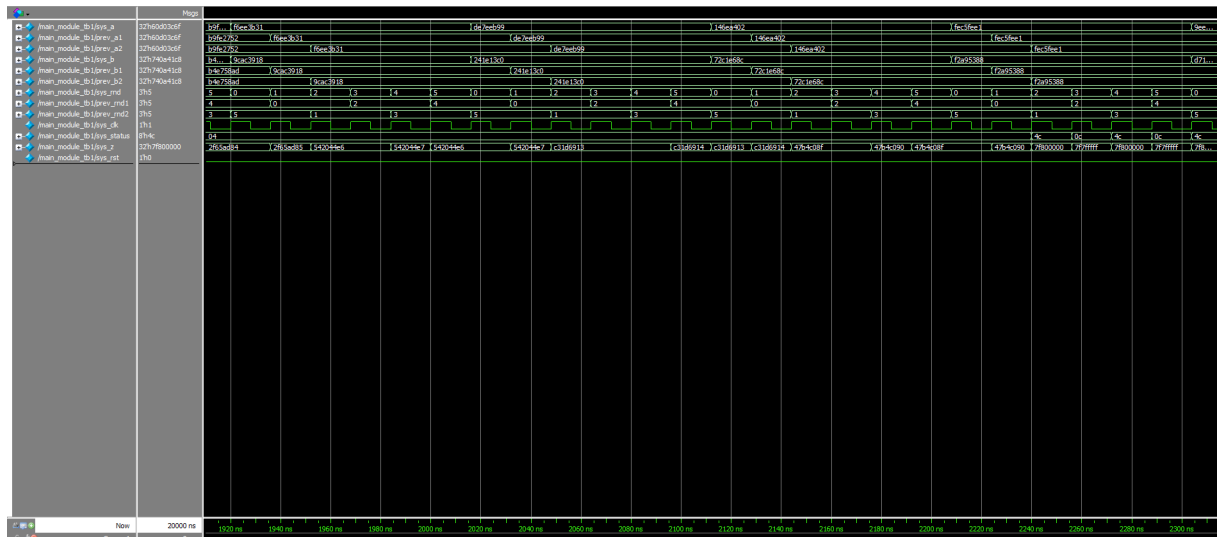
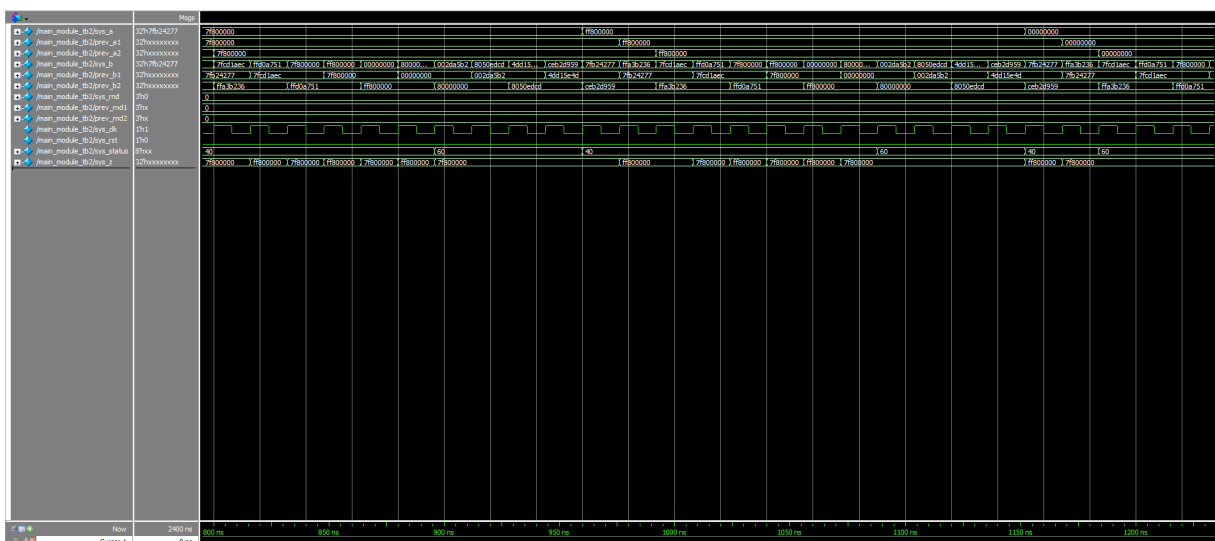
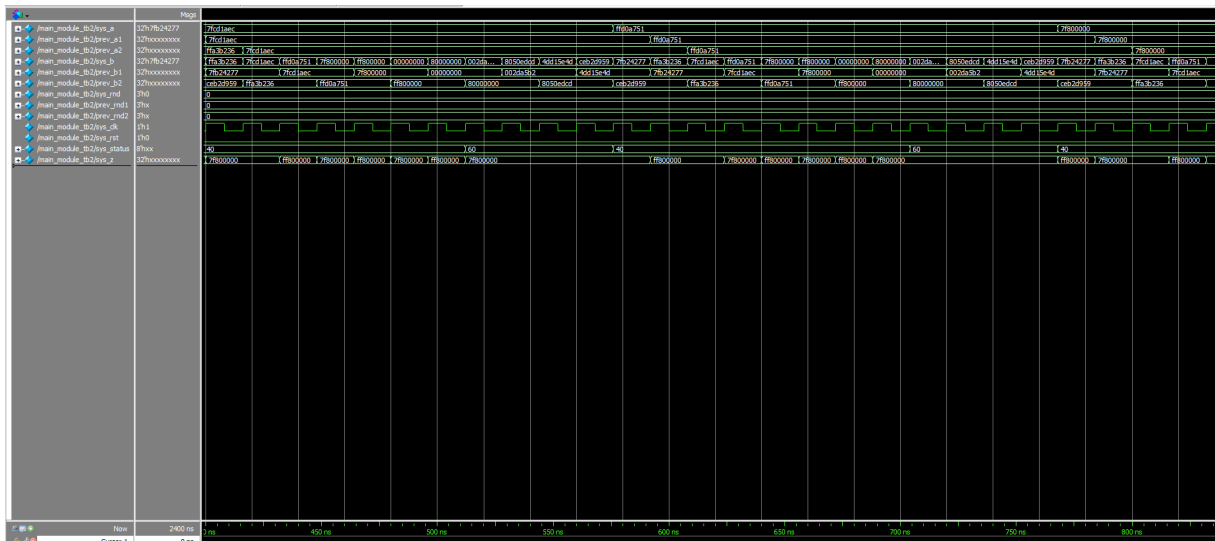
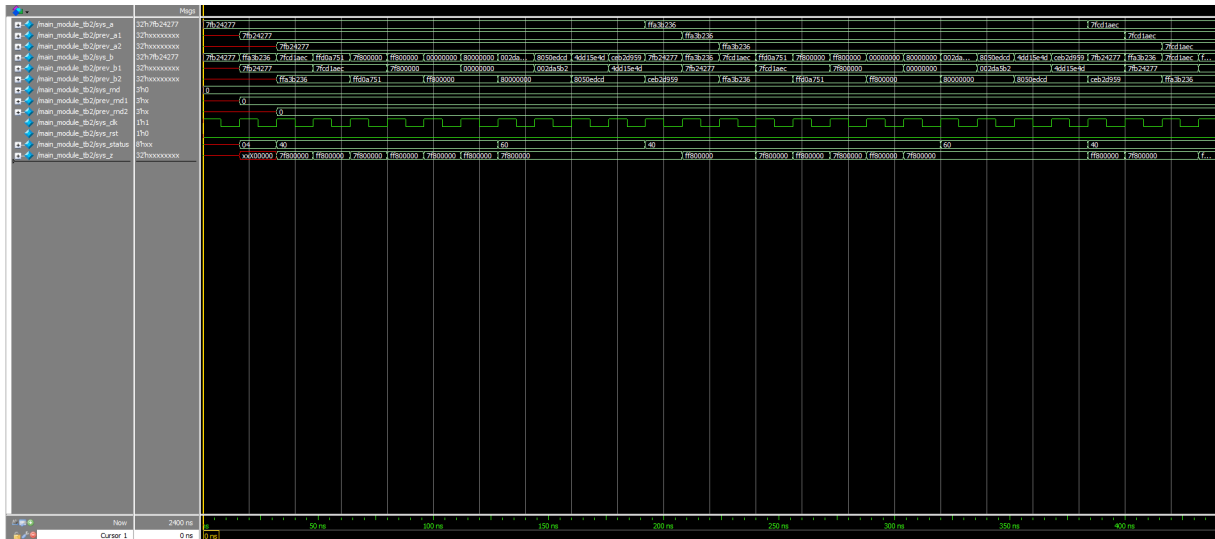


Figure 8: Διάγραμμα χρονισμού για random input

8.2 Testbench 2



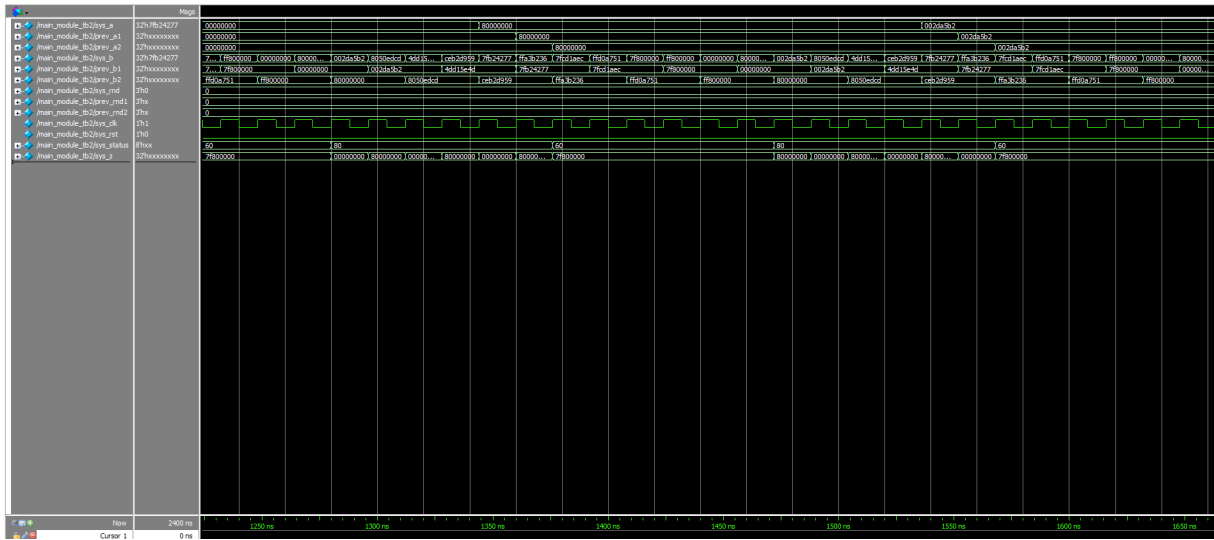


Figure 12: Διάγραμμα χρονισμού για edge cases

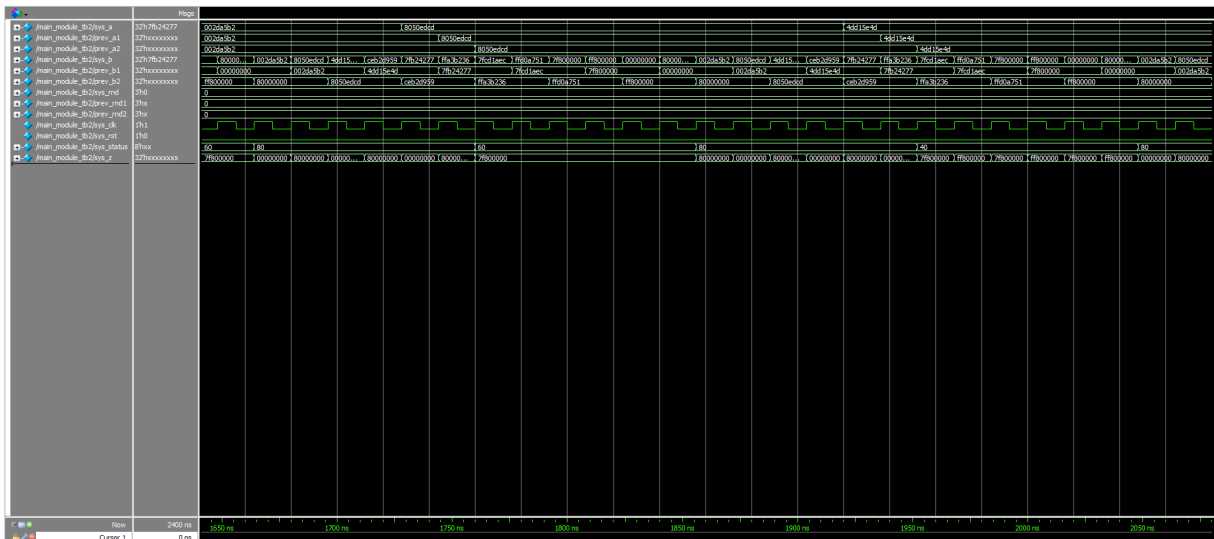


Figure 13: Διάγραμμα χρονισμού για edge cases



Figure 14: Διάγραμμα χρονισμού για edge cases